

L20: Multi-Stage Service Networks

Simulation Without a Black Box

Outline

Learning Objectives

Tandem Queues and Burke's Theorem

Jackson Networks

SimPy Implementation

Identifying the Bottleneck

3-Stage TikZ Diagram

Sensitivity Analysis

Key Takeaways

Learning Objectives

By the end of this lecture you will be able to:

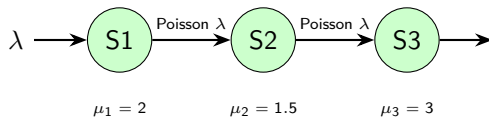
- State Burke's theorem and explain its consequence for tandem queues.
- Describe the Jackson network product-form stationary distribution.
- Implement a multi-stage network in SimPy by chaining generators.
- Identify the bottleneck stage and predict system throughput.
- Analyse sensitivity: which stage's capacity improvement matters most.

Tandem Queues and Burke's Theorem

Burke's Theorem (1956)

For an M/M/c queue in steady state:

1. The departure process is Poisson with rate λ .
2. The number in the system at time t is independent of the departure process prior to t .



Bottleneck: $\rho_2 = \lambda/\mu_2$ highest.

Consequence for tandem queues: If stage k is M/M/c, its output feeds stage $k + 1$ as a Poisson process. Each stage can be analysed independently as an M/M/c queue!

Jackson Networks: Product-Form Stationary Distribution

Jackson network (1963)

A network of K M/M/c queues with Poisson external arrivals and probabilistic routing has a product-form stationary distribution:

$$\pi(\mathbf{n}) = \prod_{k=1}^K \pi_k(n_k)$$

where each $\pi_k(n_k)$ is the marginal M/M/c stationary distribution for queue k with effective arrival rate λ_k (solved from traffic equations).

Traffic equations:

$$\lambda_k = \gamma_k + \sum_{j=1}^K \lambda_j p_{jk}, \quad k = 1, \dots, K$$

where γ_k = external arrival rate to node k , p_{jk} = routing probability from j to k .

- **Stability:** $\rho_k = \lambda_k / (c_k \mu_k) < 1$ for all k .

SimPy: Chaining Generators Across Stages

```
import simpy, numpy as np

def entity(env, stages, mus, rng, stage_waits):
    """Pass through all stages sequentially."""
    for k, (server, mu) in enumerate(zip(stages, mu)):
        arrive_k = env.now
        with server.request() as req:
            yield req
            stage_waits[k].append(env.now - arrive_k)
            yield env.timeout(rng.exponential(1/mu))

def source(env, stages, mus, lam, rng, stage_waits):
    while True:
        yield env.timeout(rng.exponential(1/lam))
        env.process(entity(env, stages, mus, rng, stage_waits))

mus = [2.0, 1.5, 3.0] # service rates per stage
stages = [simpy.Resource(env, capacity=1) for _ in mus]
stage_waits = [[] for _ in mus]
```

Identifying the Bottleneck Stage

Example: $\lambda = 1.2$, single-server stages.

Stage	μ_k	ρ_k	$W_{q,k}$
S1	2.0	0.60	0.75
S2	1.5	0.80	2.67
S3	3.0	0.40	0.22

S2 is the bottleneck: highest ρ , longest W_q .

Total $W = \sum_k (W_{q,k} + 1/\mu_k) = 4.97$ min.

Throughput bound

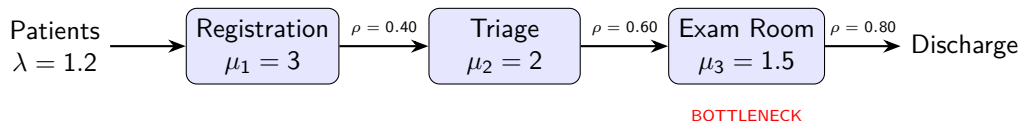
$$\text{Throughput} \leq \min_k (c_k \mu_k)$$

Adding capacity at any stage other than the bottleneck does *not* improve throughput.

Detection in simulation

Plot `server.count` / `server.capacity` over time for each stage. The consistently near-1.0 stage is the bottleneck.

Three-Stage Network: Clinic Flow



- Exam Room ($\rho = 0.80$) is the bottleneck — longest queue forms here.
- Adding a second exam room cuts $W_{q,3}$ from 2.67 to 0.44 min (Erlang-C), reducing total W by 46%.
- Speeding up Registration ($\rho = 0.40$) has negligible effect on total throughput.

Sensitivity: Which Stage's Capacity Matters Most?

Scenario: double capacity of one stage at a time ($\lambda = 1.2$).

Intervention	W (min)	Reduction
Baseline	4.97	—
$2 \times S1$ ($\mu_1 = 4$)	4.75	4.4%
$2 \times S2$ ($\mu_2 = 3$)	2.87	42.3%
$2 \times S3$ ($\mu_3 = 6$)	3.91	21.3%

Lesson

Resource allocated to the bottleneck (S2) yields the greatest system-wide benefit. This is the operational implication of bottleneck theory (TOC).

In simulation: run these as scenarios with

`simdes.analysis.scenarios.compare_scenarios` (L22).

Key Takeaways

Core ideas from L20

1. **Burke's theorem:** M/M/c departures are $\text{Poisson}(\lambda)$ — each stage of a tandem is an independent M/M/c.
2. **Jackson networks:** product-form solution; solve traffic equations for λ_k , then apply M/M/c formulas.
3. SimPy multi-stage: chain with `server.request()` blocks sequentially inside one generator.
4. The **bottleneck** is the stage with highest ρ_k ; it limits throughput.
5. **Sensitivity:** double the bottleneck's capacity first — all other improvements yield diminishing returns.

Next: L21 — The (s,S) inventory control policy: simulation and analysis.